EngineSearch-Core · Evidencias de la entrega E1

ICC006 Análisis de Algoritmos · CETYS Universidad, Mexicali

Alumno	Ari Gabriel Fernandez Villarreal
Matrícula	M045944
Equipo	Individual
Versión del código	E1
Repositorio	https://github.com/AriFerv/EngineSearch-Core
Fecha	5 de noviembre de 2026
Entorno	Python 3.13, solo biblioteca estándar

1. Contrato de datos

El programa carga un CSV UTF-8 con encabezados exactos **doc_id, title, body**. El archivo se valida completo antes de publicar un corpus; ante cualquier error se lanza *ValueError* con la línea y el motivo, y el corpus anterior permanece intacto.

Campo	Validación	Uso
doc_id	Texto no vacío; único tras strip; distingue mayúsculas y minúsculas	Identificador devuelto; las listas salen ordenadas por doc_id
title	Texto no vacío tras strip	Se tokeniza junto con el cuerpo
body	Texto UTF-8; puede estar vacío	Se tokeniza junto con el título

Se rechaza: archivo inexistente o vacío; bytes que no son UTF-8; encabezados distintos de los exigidos; filas con un número de columnas distinto de tres; comillas sin cerrar; doc_id vacío o repetido; title vacío; archivos sin ningún documento y corpus que no producen ningún token. Se tolera el BOM de UTF-8 y se ignoran las líneas completamente en blanco.

Normalización y tokenización. normalizar aplica Unicode NFKC y casefold, por lo que «PYTHON» y «python» coinciden, pero «canción» y «cancion» siguen siendo distintas. tokenizar devuelve las secuencias de letras o dígitos Unicode; el guion bajo actúa como separador y no se eliminan stopwords.

Índice y consultas. El índice invertido asocia cada término con su frecuencia total y la lista ordenada de IDs, sin repetir un ID aunque el término aparezca varias veces en el mismo documento. La búsqueda exacta acepta una sola palabra después de normalizar; una palabra ausente devuelve la lista vacía y las consultas vacías, sin tokens o con más de un token se rechazan con un mensaje claro.

2. Diagrama de la arquitectura

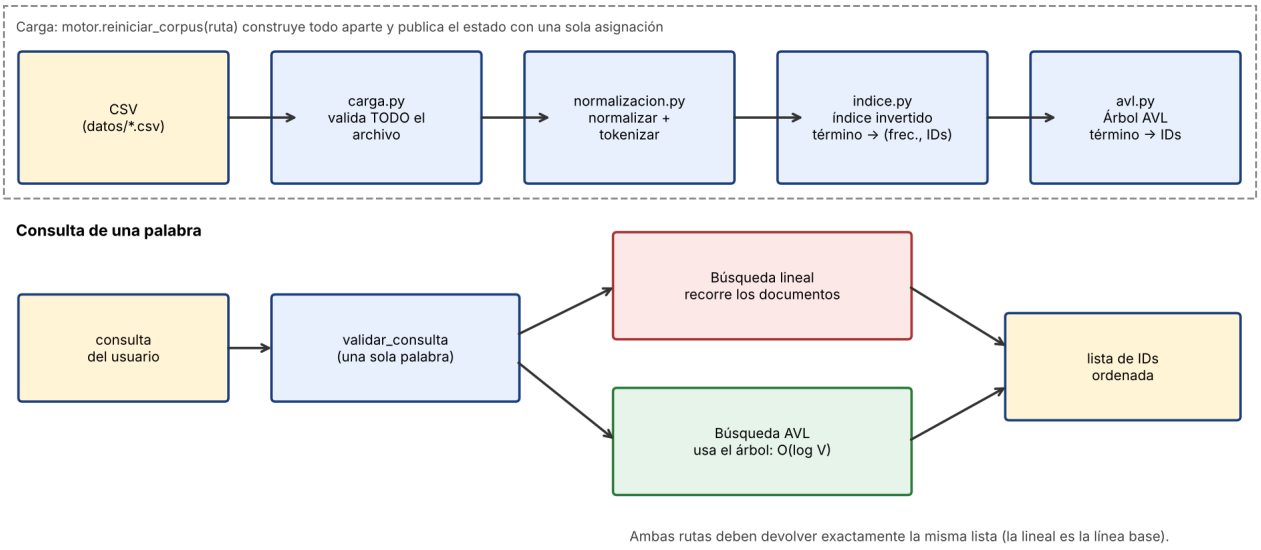


Figura 1. Flujo de carga (arriba) y de consulta (abajo). La búsqueda lineal es independiente del índice y del AVL.

Responsabilidades. carga.py valida el CSV; normalizacion.py normaliza y tokeniza; indice.py construye el índice y contiene la búsqueda lineal; avl.py implementa el árbol; motor.py mantiene el estado del corpus y lo reinicia de forma atómica; main.py y pruebas/interfaz_evaluacion.py son las interfaces.

3. Traza del árbol AVL

Convenciones: altura(vacío) = 0 y altura(hoja) = 1; balance = altura(izq) − altura(der). El invariante exige balance en {−1, 0, 1} en todos los nodos, además del orden de búsqueda.

Inserción de 30, 20, 10 (caso LL)

Paso	Acción	Estado y balances
1	Insertar 30	Raíz 30; altura 1; balance 0.
2	Insertar 20	20 queda como hijo izquierdo de 30; altura(30) = 2; balance(30) = +1, se permite.
3	Insertar 10	10 queda como hijo izquierdo de 20; altura(10) = 1, altura(20) = 2, altura(30) = 3; balance(30) = 2 − 0 = +2: se viola el invariante. El hijo izquierdo (20) tiene balance +1, es el caso LL.
4	Rotación a la derecha en 30	20 sube a la raíz; 30 pasa a ser su hijo derecho. Resultado: raíz 20 con hijos 10 y 30, altura 2 y balance 0 en todos los nodos.

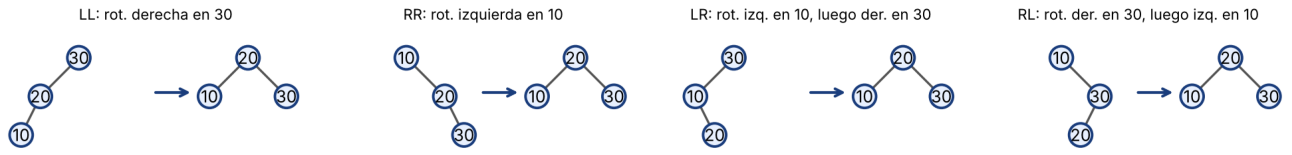


Figura 2. Los cuatro casos de rotación. En los cuatro la raíz final es 20 con hijos 10 y 30 (prueba T08).

Construcción con el vocabulario del corpus de referencia

Al insertar en orden alfabético los 35 términos del corpus de referencia, el árbol tiene altura **6** y raíz «eficiencia»; se hicieron las rotaciones RR: 29. verificar() devuelve **verdadero**. La búsqueda de «busca» recorre 4 nodos: eficiencia → cuesta → balanceados → busca; la búsqueda lineal revisa los 6 documentos completos en cada consulta.

4. Tabla esperado / obtenido

Los valores de la columna «Obtenido» se generaron ejecutando el código de la versión E1 (no están escritos a mano). La suite completa (39 pruebas, T01–T08 y T14–T16) se ejecutó con *python -m unittest discover -s pruebas* y terminó en OK con Python 3.13.7. Versión del código: E1 en todas las filas.

ID	Req.	Entrada o acción	Esperado	Obtenido	Análisis	Estado
T01	RT03	Cargar documentos_referencia.csv	6 IDs únicos: D01 a D06	6 IDs únicos: ['D01', 'D02', 'D03', 'D04', 'D05', 'D06']	Los 6 IDs son únicos: el contrato acepta el corpus oficial.	**OK**
T02	RT03, RT04, RT07	Buscar «busca» (lineal y AVL)	['D01', 'D02', 'D03']; lineal = AVL	lineal ['D01', 'D02', 'D03']; AVL ['D01', 'D02', 'D03']	Lineal y AVL coinciden; el AVL no leyó ningún documento.	**OK**
T03	RT02, RT03	Buscar «palabras» (lineal y AVL)	['D02', 'D03', 'D05', 'D06']; frecuencia total 4	lineal ['D02', 'D03', 'D05', 'D06']; AVL ['D02', 'D03', 'D05', 'D06']; frecuencia 4	La frecuencia (4) cuenta apariciones; el posteo lista documentos.	**OK**
T04	RT03	Buscar «python» (lineal y AVL)	['D01']; frecuencia total 2	lineal ['D01']; AVL ['D01']; frecuencia 2	«python» aparece 2 veces, pero D01 se lista una sola vez.	**OK**
T05	RT04, RT07	Buscar «inexistente» (sin Bloom)	[] en lineal y AVL; sin excepción	lineal []; AVL []	Ausente = lista vacía, no excepción; aún sin Bloom.	**OK**
T06	RT01, RT02	«PYTHON» vs «python»; «canción» vs «cancion» (corpus_extra.csv)	PYTHON = python → [D01]; canción → [M01]; cancion → [M02]	PYTHON ['D01'], python ['D01']; canción ['M01']; cancion ['M02']	Casefold unifica mayúsculas; NFKC conserva ó y o como palabras distintas.	**OK**
T07	RT05	Cargar cada CSV de datos/invalidos/ con un corpus ya cargado	ValueError con diagnóstico en cada uno; corpus previo intacto	8 de 8 rechazados. ids_duplicados.csv → «Línea 4: doc_id duplicado 'D01'.». Corpus previo: 6 documentos, 35 términos, búsquedas iguales.	Todos se rechazan con línea y motivo; el estado previo no cambió (carga atómica).	**OK**
T08	RT06 a RT09	Insertar 30,20,10 (LL); 10,20,30 (RR); 30,10,20 (LR); 10,30,20 (RL)	Cada caso hace su rotación; raíz 20; preorden [20, 10, 30]; verificar() verdadero; in-order [10, 20, 30]	LL: [30, 20, 10] → rot. ['LL'], raíz 20, preorden [20, 10, 30]; RR: [10, 20, 30] → rot. ['RR'], raíz 20, preorden [20, 10, 30]; LR: [30, 10, 20] → rot. ['LR'], raíz 20, preorden [20, 10, 30]; RL: [10, 30, 20] → rot. ['RL'], raíz 20, preorden [20, 10, 30]	Cada caso hace solo su rotación y termina con raíz 20; verificar() confirma el invariante.	**OK**
T14	RT05	Cargar la referencia y luego un corpus de un documento (X01)	Sin términos ni IDs residuales	IDs ['X01']; vocabulario ['duerme', 'el', 'gato', 'negro']; «python» y «busca» → []	No quedan términos del corpus anterior: el estado se reemplaza completo.	**OK**
T15	RT05	Cargar archivo vacío, solo encabezado y corpus sin tokens	Rechazo con motivo; corpus previo intacto	archivo vacío → «El archivo 'datos/vacio.csv' está vacío.» | solo encabezado → «El archivo no contiene ningún documento.» | sin tokens → «El corpus no produce ningún token (solo letras o dígitos cuentan); se rechaza.»	Rechazo seguro con motivo, sin perder el corpus cargado.	**OK**
T16	RT01, RT02	Consultas «», «dos palabras» y «!!!»	Mensaje claro, sin caída; el motor sigue funcionando	'' → «La consulta está vacía.» | 'dos palabras' → «La búsqueda exacta acepta una sola palabra; la consulta contiene 2: ['dos', 'palabras'].» | '!!!' → «La consulta no contiene ninguna palabra (solo cuentan letras y dígitos).»	El error es un ValueError con mensaje; el programa no cae.	**OK**

5. Altura, invariante y complejidad

Invariante. En un árbol AVL, para cualquier nodo, la diferencia entre la altura de su subárbol izquierdo y la de su subárbol derecho es como máximo 1 en valor absoluto (balance −1, 0 o 1); el signo indica hacia qué lado se inclina el nodo. Además se conserva el orden de búsqueda. Si una inserción rompe el invariante, se restaura con una rotación (LL, RR, LR o RL).

Altura. Sea N(h) el mínimo de nodos de un AVL de altura h. El árbol más flaco tiene una raíz, un subárbol de altura h−1 y otro de altura h−2, así que $N(h) = 1 + N(h-1) + N(h-2)$, con $N(1) = 1$ y $N(2) = 2$ (1, 2, 4, 7, 12, 20...). Ese mínimo crece de forma exponencial con la altura, por lo que un AVL con n nodos no puede ser muy alto: $h < 1.4405 \cdot \log_2(n+2)$ (Cormen et al., 2022). En el corpus de referencia, con n = 35 términos, la altura obtenida es 6, menor que la cota de 7.5.

Complejidad. Buscar o insertar en el AVL recorre un camino de la raíz a una hoja, que mide como máximo la altura, así que cuesta O(log V) con V términos distintos (más O(r) para devolver una lista de r IDs). La búsqueda lineal tokeniza todos los documentos en cada consulta, Θ(T) con T tokens, y no usa el índice. La diferencia crece con el tamaño del corpus: el índice paga su construcción una sola vez y las consultas dejan de depender del número de documentos.

Resumen de costos de las operaciones de E1 (T = total de tokens del corpus, V = términos distintos, r = tamaño de la lista devuelta):

Operación	Costo	Nota
Cargar y validar el CSV	O(tamaño del archivo)	Se lee y valida completo antes de publicar.
construir_indice	O(T) + O(V log V) + ordenar cada lista de IDs	Recorre los tokens, ordena el vocabulario y los posteos.
buscar_lineal (por consulta)	Θ(T)	Tokeniza todos los documentos en cada consulta.
Construir el AVL con los V términos	O(V log V)	V inserciones de O(log V) cada una.
AVL: insertar	O(log V)	Altura acotada por el invariante.
AVL: buscar (por consulta)	O(log V + r)	O(log V) para localizar; O(r) para devolver la lista.

Bibliografía

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

González Escandón, R. (2026). *Análisis de algoritmos con Python: Complejidad, estructuras y paradigmas* (1.ª ed.). Publicación independiente.